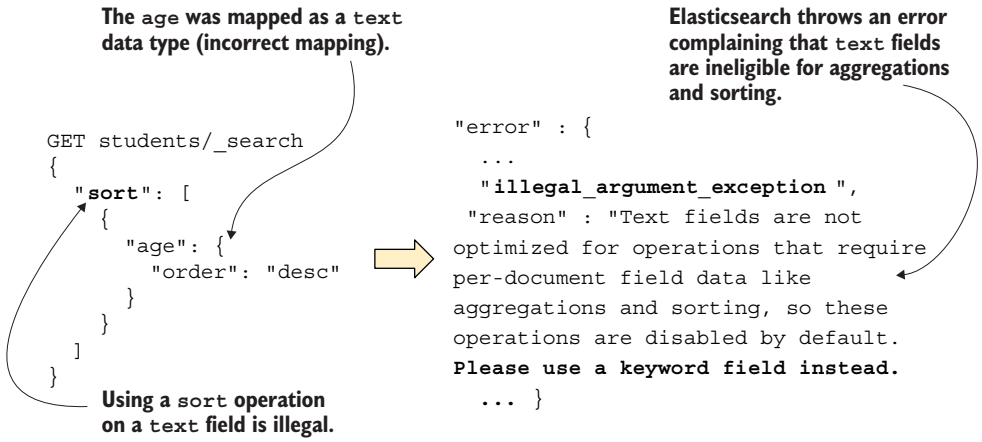


Figure 4.4 A sort operation on a text field results in an error.

If we try to sort on a `text` field, Elasticsearch throws an error message saying that the field isn't optimized for sorting. If we want to override this behavior and sort `text` fields, we can do so by setting `fielddata` to `true` when defining the schema:

```
PUT students_with_fielddata_setting
{
  "mappings": {
    "properties": {
      "age": {
        "type": "text",
        "fielddata": true
      }
    }
  }
}
```

A word of advice: field data is stored on the cluster's heap memory in a *field data cache*. Enabling `fielddata` leads to expensive computations, which may in turn lead to performance issues on the cluster. I suggest using `keyword` fields rather than enabling `fielddata` (see the next section).

FIXING SORTING USING KEYWORD TYPES

Fortunately, Elasticsearch can create any field as a multi-field with `keyword` as the second type by default. Going with this default feature, the `age` field is tagged as a multi-field, thus creating `age.keyword` with `keyword` as the data type. To sort students, all we need to do is change the field name from `age` to `age.keyword`.

Listing 4.3 Modified `sort` query sorting on the `age.keyword` field

```
GET students_temp/_search
{
```

Uses the _search API to fetch all documents